Programmation Frontend

Lab — Des fonctions JS à React

Fonctions, asynchrone et votre première application React

ESP/UCAD — 2025–2026

Dr. El Hadji Bassirou TOURÉ
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Informations pratiques

Durée estimée	1h (une séance)
Prérequis	Savoir manipuler le DOM (sélecteurs, événements, modification de styles). Avoir un éditeur de code (VS Code recommandé) et Node.js installé.
Fichiers starter	`index.html` + `index.js` (fournis par l'enseignant)
Livrables	Un dossier `annuaire-react/` fonctionnel avec une appli React
Validation	L'application React affiche des utilisateurs récupérés depuis une API

Objectifs du lab

À l'issue de ce lab, chaque étudiant sera capable de :

1. Distinguer les **trois façons de déclarer une fonction** en JavaScript (déclaration, expression, fléchée)
2. Expliquer ce qu'est une opération **asynchrone** et pourquoi c'est nécessaire
3. Utiliser `fetch` avec `async/await` pour récupérer des données depuis une API
4. Créer une application **React** avec Vite et comprendre le concept de **composant**
5. Utiliser `useState` et `useEffect` pour gérer l'état et les effets de bord

Table des matières

1 Le starter — Point de départ **3**
1.1 Le fichier `index.html` . . . 3
1.2 Le fichier `index.js` . . . 3
1.3 Testez le starter . . . 4

2 Partie 1 — Les trois visages des fonctions **4**
2.1 Façon 1 : La déclaration de fonction . . . 4
2.2 Façon 2 : L'expression de fonction . . . 4
2.3 Façon 3 : La fonction fléchée (*arrow function*) . . . 5
2.4 Exercice 1 — Refactoriser le starter . . . 5

3 Partie 2 — JavaScript asynchrone **7**
3.1 Le problème : attendre sans bloquer . . . 7
3.2 L'API JSONPlaceholder . . . 7
3.3 Récupérer des données avec `fetch` . . . 8
3.3.1 Méthode 1 : `.then()` (pour comprendre) . . . 8
3.3.2 Méthode 2 : `async/await` (la méthode moderne) . . . 8
3.4 Exercice 2 — Afficher des utilisateurs dans le DOM . . . 9

4 Partie 3 — Bienvenue dans React **11**
4.1 Le problème du DOM manuel . . . 11
4.2 Installer Vite + React . . . 11
4.3 Comprendre la structure du projet . . . 12
4.4 Votre premier composant React . . . 12
4.5 Ajouter de l'état avec `useState` . . . 12
4.6 Exercice 3 — L'annuaire en React . . . 13
4.7 Comparer les deux versions . . . 15

5 Aide-mémoire **15**

6 Résumé et conclusion **16**

1 Le starter — Point de départ

Créez un dossier `lab-react/` et placez-y les deux fichiers suivants.

1.1 Le fichier `index.html`

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0" />
    <title>DOM</title>
    <script src="index.js"></script>
  </head>
  <body>
    <h2>Manipulation du DOM</h2>
    <p>Premier exemple</p>
    <input type="button" value="changer couleur" id="b1" />
  </body>
</html>
```

La page contient un titre `<h2>`, un paragraphe `<p>` et un bouton `<input>` avec l'identifiant `b1`. Le fichier `index.js` est chargé via la balise `<script>`.

1.2 Le fichier `index.js`

```
"use strict";
(function () {
  window.addEventListener("load", init);

  function init() {
    id("b1").addEventListener("click", change);
  }

  function change() {
    let p = qs("p");
    let h2 = qs("h2");
    p.style.color = "red";
    h2.style.color = "blue";
  }

  function id(name) {
    return document.getElementById(name);
  }
  function qs(selector) {
    return document.querySelector(selector);
  }
  function qsa(query) {
    return document.querySelectorAll(query);
  }
})();
```

Le JS est encapsulé dans une IIFE[1]. Il contient trois fonctions utilitaires (`id`, `qs`, `qsa`), une fonction `init` qui attache un écouteur de clic au bouton, et une fonction `change` qui modifie les

[1] IIFE = *Immediately Invoked Function Expression* : une fonction qui s'exécute immédiatement.

couleurs.

1.3 Testez le starter

Étape 0 — Vérification

Ouvrez `index.html` dans votre navigateur. Cliquez sur le bouton « changer couleur ». Le titre doit passer en **bleu** et le paragraphe en **rouge**.
Si ça ne fonctionne pas : vérifiez que les deux fichiers sont dans le **même dossier** et que les noms correspondent (`index.html` charge `index.js`).

Une fois que ça marche, on va **enrichir ce code** en trois étapes : d'abord les fonctions, puis l'asynchrone, et enfin React.

2 Partie 1 — Les trois visages des fonctions

Objectif — 15 minutes

Comprendre les trois façons d'écrire une fonction en JavaScript et savoir quand utiliser chacune.

2.1 Façon 1 : La déclaration de fonction

C'est celle que vous connaissez déjà. Regardez le starter :

```
function change() {
  let p = qs("p");
  p.style.color = "red";
}
```

On déclare une fonction avec le mot-clé `function`, suivi d'un nom. C'est la forme classique.

Particularité : le *hoisting*

Une fonction déclarée peut être appelée **avant** sa définition dans le code. JavaScript la « remonte » automatiquement en haut du fichier. C'est pour cela que dans le starter, `init()` appelle `change()` qui est définie plus bas — et ça marche.

2.2 Façon 2 : L'expression de fonction

On peut aussi **stocker une fonction dans une variable**, comme on stocke un nombre ou un texte :

```
const change = function() {
  let p = qs("p");
  p.style.color = "red";
};
```

La différence ? Ici, `change` est une **variable** qui contient une fonction. On ne peut **pas** l'appeler avant sa définition (pas de *hoisting*).

À retenir

Déclaration	`function maFonction() { ... }`
Expression	`const maFonction = function() { ... };`

En JavaScript, **une fonction est une valeur** comme une autre. On peut la stocker dans une variable, la passer en paramètre, la retourner. C'est un concept fondamental.

2.3 Façon 3 : La fonction fléchée (*arrow function*)

Depuis 2015 (ES6), JavaScript offre une syntaxe plus courte :

```
const change = () => {
  let p = qs("p");
  p.style.color = "red";
};
```

On remplace `function()` par `() =>`. C'est la même chose, en plus compact. La flèche se lit « qui fait » : `() => { ... }` = « une fonction sans paramètre qui fait ... ».

Quand la fonction a **un seul paramètre**, on peut enlever les parenthèses. Quand elle a **une seule ligne**, on peut enlever les accolades et le `return` :

```
// Avec un paramètre et une seule ligne
const id = name => document.getElementById(name);

// Équivalent de :
function id(name) {
  return document.getElementById(name);
}
```

Les trois formes côte à côte

Forme	Syntaxe
Déclaration	`function double(x) { return x * 2; }`
Expression	`const double = function(x) { return x * 2; };`
Fléchée	`const double = x => x * 2;`

Les trois font **la même chose**. La fonction fléchée est la plus utilisée dans le code moderne et dans React.

2.4 Exercice 1 — Refactoriser le starter

À vous de jouer

Réécrivez les fonctions utilitaires du starter en utilisant des **fonctions fléchées**. Transformez :

```
// AVANT (déclarations classiques)
function id(name) {
  return document.getElementById(name);
```

```
}
function qs(selector) {
  return document.querySelector(selector);
}
function qsa(query) {
  return document.querySelectorAll(query);
}
```

En :

```
// APRÈS (fonctions fléchées)
const id = name => document.getElementById(name);
const qs = selector => document.querySelector(selector);
const qsa = query => document.querySelectorAll(query);
```

Remplacez aussi la fonction `change` par une fléchée, et `init` par une fléchée. Votre starter refactorisé doit ressembler à ceci :

```
"use strict";
(function () {
  window.addEventListener("load", init);

  const init = () => {
    id("b1").addEventListener("click", change);
  };

  const change = () => {
    qs("p").style.color = "red";
    qs("h2").style.color = "blue";
  };

  const id = name => document.getElementById(name);
  const qs = selector => document.querySelector(selector);
  const qsa = query => document.querySelectorAll(query);
})();
```

Attention au *hoisting*

Si vous testez ce code, il ne fonctionnera **pas** directement ! Pourquoi ? Parce que `init` est maintenant une expression (pas de *hoisting*), mais `window.addEventListener("load", init)` est appelé avant la définition de `init`.
La solution : déplacez l'appel `window.addEventListener` **après** les définitions :

```
"use strict";
(function () {
  const id = name => document.getElementById(name);
  const qs = selector => document.querySelector(selector);
  const qsa = query => document.querySelectorAll(query);

  const change = () => {
    qs("p").style.color = "red";
    qs("h2").style.color = "blue";
  };

  const init = () => {
    id("b1").addEventListener("click", change);
```

```
  };

  window.addEventListener("load", init);
})();
```

Avec les fonctions fléchées (expressions), **l'ordre des définitions compte**. Pas de *hoisting*.

Sauvegardez `index.js` et rechargez `index.html` dans le navigateur. Le bouton doit fonctionner exactement comme avant.

3 Partie 2 — JavaScript asynchrone

Objectif — 20 minutes

Comprendre le concept d'asynchrone et savoir récupérer des données depuis une API avec `fetch` et `async/await`.

3.1 Le problème : attendre sans bloquer

Imaginez que vous commandez un thiéboudienne au restaurant. Deux scénarios :

	Synchrone	**Asynchrone**
Ce qui se passe	Vous restez debout devant la cuisine, immobile, à attendre que le plat soit prêt.	Vous vous asseyez, consultez votre téléphone. On vous appelle quand c'est prêt.
Résultat	Vous êtes bloqué. Vous ne pouvez rien faire d'autre.	Vous continuez votre vie. Le plat arrive quand il est prêt.

JavaScript fonctionne en mode **asynchrone**. Quand on demande des données à un serveur (ce qui prend du temps), JavaScript **n'attend pas**. Il continue d'exécuter le reste du code, et traite les données **quand elles arrivent**.

À retenir

Synchrone = une chose à la fois, dans l'ordre, en attendant chaque résultat.
Asynchrone = on lance une tâche, on continue autre chose, et on traite le résultat quand il arrive.
JavaScript est **mono-thread** (un seul fil d'exécution). Sans l'asynchrone, la page se figerait à chaque requête réseau. C'est pour cela que l'asynchrone est **indispensable** en développement web.

3.2 L'API JSONPlaceholder

Pour pratiquer, on va utiliser une API gratuite de test : `https://jsonplaceholder.typicode.com`. Elle fournit de fausses données (utilisateurs, articles, commentaires) au format JSON.

L'adresse `https://jsonplaceholder.typicode.com/users` retourne 10 utilisateurs fictifs. Ouvrez cette URL dans votre navigateur pour voir les données.

Un utilisateur ressemble à ceci :

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz",
  "phone": "1-770-736-8031 x56442",
  "company": {
    "name": "Romaguera-Crona"
  }
}
```

3.3 Récupérer des données avec `fetch`

La fonction `fetch` envoie une requête HTTP et retourne une **promesse** (*Promise*) — c'est-à-dire un objet qui représente un résultat qui **n'est pas encore disponible**.

Qu'est-ce qu'une promesse ?

Quand vous commandez votre thiéboudienne, le serveur vous donne un **ticket avec un numéro**. Ce ticket est une *promesse* : il ne contient pas le plat, mais il **garantit** que le plat arrivera (ou qu'on vous préviendra s'il y a un problème). En JavaScript, `fetch` retourne un tel ticket.

3.3.1 Méthode 1 : `.then()` (pour comprendre)

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())  // Convertir la réponse en JSON
  .then(users => {
    console.log(users);               // Afficher les utilisateurs
  })
  .catch(error => {
    console.error("Erreur :", error);  // Gérer les erreurs
  });
```

Chaque `.then()` reçoit le résultat de l'étape précédente. C'est une chaîne : requête → conversion en JSON → utilisation des données.

3.3.2 Méthode 2 : `async/await` (la méthode moderne)

La syntaxe `async/await` fait exactement la même chose, mais en **plus lisible** :

```
const chargerUtilisateurs = async () => {
  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );
    const users = await response.json();
    console.log(users);
  } catch (error) {
    console.error("Erreur :", error);
  }
};
```

Décoder `async/await`

Mot-clé	Signification
`async`	Devant une fonction : « cette fonction contient des opérations asynchrones »
`await`	Devant une promesse : « attends le résultat avant de continuer »
`try/catch`	« Essaie ceci, et si ça échoue, fais cela »

`await` ne bloque **pas** le navigateur. Il met la fonction en pause et laisse JavaScript continuer d'autres tâches. Quand la réponse arrive, la fonction reprend.

3.4 Exercice 2 — Afficher des utilisateurs dans le DOM

À vous de jouer

Créez un fichier HTML + JS qui récupère les utilisateurs depuis JSONPlaceholder et les affiche dans la page. Utilisez le patron du starter.

Créez un fichier `annuaire.html` dans votre dossier `lab-react/` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Annuaire</title>
  <style>
    body { font-family: sans-serif; max-width: 600px; margin: 2rem auto; }
    .card { border: 1px solid #ccc; padding: 1rem; margin: 0.5rem 0;
            border-radius: 8px; }
    .card h3 { margin: 0 0 0.5rem 0; }
    .card p { margin: 0.2rem 0; color: #555; }
    #btn-charger { padding: 0.5rem 1.5rem; font-size: 1rem;
                   cursor: pointer; }
  </style>
</head>
<body>
  <h1>Annuaire des utilisateurs</h1>
  <button id="btn-charger">Charger les utilisateurs</button>
  <div id="liste"></div>
  <script src="annuaire.js"></script>
</body>
</html>
```

Maintenant, créez `annuaire.js` dans le même dossier, en suivant le patron du starter :

```
"use strict";
(function () {
  const id = name => document.getElementById(name);

  // Fonction asynchrone qui récupère et affiche les utilisateurs
  const chargerUtilisateurs = async () => {
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
```

```
      );
      const users = await response.json();
      afficherUtilisateurs(users);
    } catch (error) {
      id("liste").textContent = "Erreur de chargement.";
      console.error("Erreur :", error);
    }
  };

  // Fonction qui construit le HTML pour chaque utilisateur
  const afficherUtilisateurs = (users) => {
    const liste = id("liste");
    liste.innerHTML = "";  // Vider la liste avant de remplir

    users.forEach(user => {
      const card = document.createElement("div");
      card.classList.add("card");
      card.innerHTML = `
        <h3>${user.name}</h3>
        <p>Email : ${user.email}</p>
        <p>Entreprise : ${user.company.name}</p>
      `;
      liste.appendChild(card);
    });
  };

  const init = () => {
    id("btn-charger").addEventListener("click", chargerUtilisateurs);
  };

  window.addEventListener("load", init);
})();
```

Ouvrez `annuaire.html` dans votre navigateur. Cliquez sur le bouton. Les 10 utilisateurs apparaissent.

Vérification

Si rien ne s'affiche, ouvrez la **console du navigateur** (`F12` → onglet Console). Les erreurs y apparaissent en rouge. Causes fréquentes :

- Erreur de syntaxe dans le JS (guillemet manquant, parenthèse oubliée)
- Fichier `annuaire.js` pas au même endroit que le HTML
- Pas de connexion internet (l'API est en ligne)

Bilan de la Partie 2

Vous savez maintenant :

1. Pourquoi l'asynchrone est nécessaire (ne pas bloquer le navigateur)
2. Utiliser `fetch` + `async/await` pour récupérer des données
3. Construire du HTML dynamiquement à partir de données JSON

Mais regardez votre code : pour afficher 10 cartes, il faut `createElement`, `innerHTML`, `appendChild`, `classList.add`... C'est **beaucoup de manipulation manuelle du DOM**. Et si on veut ajouter un champ de recherche ? Un tri ? Un filtre ? Le code devient vite ingérable. React résout ce problème.

4 Partie 3 — Bienvenue dans React

Objectif — 25 minutes

Comprendre pourquoi React existe, créer un projet React avec Vite, et recréer l'annuaire en React.

4.1 Le problème du DOM manuel

Dans l'exercice précédent, vous avez écrit du code comme :

```
const card = document.createElement("div");
card.classList.add("card");
card.innerHTML = `<h3>${user.name}</h3>`;
liste.appendChild(card);
```

C'est du code **impératif** : vous dites au navigateur **comment** faire, étape par étape. Avec React, on passe en mode **déclaratif** : on décrit **ce qu'on veut voir**, et React s'occupe de mettre à jour le DOM.

Impératif vs Déclaratif

	Impératif (DOM manuel)	**Déclaratif** (React)
Analogie	Vous expliquez au cuisinier chaque geste : « coupe l'oignon, fais chauffer l'huile, ajoute le riz... »	Vous commandez « un thiéboudienne » et le cuisinier sait quoi faire.
Code	`createElement`, `appendChild`, `innerHTML`	On décrit le résultat souhaité, React gère le DOM.

4.2 Installer Vite + React

Vite est un outil qui crée et configure un projet React en une seule commande. Il est rapide, moderne et utilisé en production.

Ouvrez un terminal et tapez :

```
npm create vite@latest annuaire-react -- --template react
```

```
cd annuaire-react
npm install
npm run dev
```

Résultat attendu :

```
  VITE v6.x.x  ready in xxx ms

  -> Local:   http://localhost:5173/
```

Ouvrez `http://localhost:5173` dans votre navigateur. Vous voyez la page d'accueil de Vite + React.

Node.js obligatoire

Si la commande `npm` n'est pas reconnue, Node.js n'est pas installé. Téléchargez-le sur https://nodejs.org (version LTS). Après l'installation, fermez et rouvrez le terminal.

4.3 Comprendre la structure du projet

Ouvrez le dossier `annuaire-react` dans VS Code. Voici les fichiers importants :

Fichier	Rôle
`src/App.jsx`	Le composant principal — c'est **ici** qu'on va travailler
`src/main.jsx`	Le point d'entrée — relie React au HTML (ne pas toucher)
`index.html`	La page HTML (contient juste un `<div id="root">`)
`package.json`	La liste des dépendances du projet

4.4 Votre premier composant React

Ouvrez `src/App.jsx` et remplacez **tout le contenu** par :

```
const App = () => {
  return (
    <div>
      <h1>Annuaire des utilisateurs</h1>
      <p>Bienvenue dans mon premier composant React !</p>
    </div>
  );
};

export default App;
```

Sauvegardez. La page dans le navigateur se met à jour **automatiquement** (c'est le *Hot Module Replacement* de Vite — pas besoin de rafraîchir).

Qu'est-ce qu'un composant ?

Un **composant React** est une fonction qui retourne du **JSX** — un mélange de JavaScript et de HTML. C'est la brique de base de toute application React. Chaque élément visible de l'interface (un bouton, une carte, une liste) est un composant.
Remarquez : on utilise une **fonction fléchée**. Tout ce que vous avez appris en Partie 1 est utilisé ici.

4.5 Ajouter de l'état avec `useState`

On veut que le bouton déclenche le chargement des utilisateurs. Pour cela, on a besoin d'un **état** : une variable que React **surveille**. Quand l'état change, React met à jour l'affichage automatiquement.

```
import { useState } from "react";

const App = () => {
  const [compteur, setCompteur] = useState(0);
```

```
  return (
    <div>
      <h1>Annuaire des utilisateurs</h1>
      <p>Vous avez cliqué {compteur} fois.</p>
      <button onClick={() => setCompteur(compteur + 1)}>
        Cliquer
      </button>
    </div>
  );
};

export default App;
```

Testez : chaque clic incrémente le compteur. La page se met à jour **sans que vous touchiez au DOM**. Pas de `innerHTML`, pas de `createElement`. Vous décrivez le résultat, React s'occupe du reste.

Décoder `useState`

Élément	Signification
`useState(0)`	Crée un état initialisé à `0`
`compteur`	La valeur actuelle de l'état
`setCompteur`	La fonction pour modifier l'état
`onClick={...}`	L'événement clic (équivalent de `addEventListener`)
`() =>`	Une fonction fléchée (Partie 1 !)

Règle d'or : ne modifiez **jamais** l'état directement (`compteur = 5`). Utilisez **toujours** la fonction `set...` fournie par `useState`.

4.6 Exercice 3 — L'annuaire en React

À vous de jouer — Exercice final

Recréez l'annuaire de la Partie 2, mais cette fois en React. L'application doit :

1. Afficher un bouton « Charger les utilisateurs »
2. Au clic, récupérer les données depuis JSONPlaceholder (`async/await`)
3. Afficher chaque utilisateur dans une carte

Remplacez le contenu de `src/App.jsx` par :

```
import { useState } from "react";

const App = () => {
  const [users, setUsers] = useState([]);
  const [chargement, setChargement] = useState(false);

  const chargerUtilisateurs = async () => {
    setChargement(true);
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );
      const data = await response.json();
```

```
      setUsers(data);
    } catch (error) {
      console.error("Erreur :", error);
    }
    setChargement(false);
  };

  return (
    <div style={{ maxWidth: "600px", margin: "2rem auto",
                  fontFamily: "sans-serif" }}>
      <h1>Annuaire des utilisateurs</h1>
      <button onClick={chargerUtilisateurs}
              disabled={chargement}>
        {chargement ? "Chargement..." : "Charger les utilisateurs"}
      </button>

      {users.map(user => (
        <UserCard key={user.id} user={user} />
      ))}
    </div>
  );
};

const UserCard = ({ user }) => {
  return (
    <div style={{ border: "1px solid #ccc", padding: "1rem",
                  margin: "0.5rem 0", borderRadius: "8px" }}>
      <h3 style={{ margin: "0 0 0.5rem 0" }}>{user.name}</h3>
      <p style={{ margin: "0.2rem 0", color: "#555" }}>
        Email : {user.email}
      </p>
      <p style={{ margin: "0.2rem 0", color: "#555" }}>
        Entreprise : {user.company.name}
      </p>
    </div>
  );
};

export default App;
```

Sauvegardez et vérifiez dans le navigateur.

Ce qui est nouveau ici

Concept	Explication
`users.map(...)`	Parcourt le tableau et crée un élément JSX pour chaque utilisateur (remplace la boucle `forEach` + `appendChild`)
`key={user.id}`	Identifiant unique que React utilise pour optimiser les mises à jour
`<UserCard />`	Un **deuxième composant** ! On découpe l'interface en morceaux réutilisables
`{ user }` dans les props	On passe des données d'un composant parent à un composant enfant
`disabled={chargement}`	Le bouton se désactive pendant le chargement

4.7 Comparer les deux versions

Prenez un moment pour comparer :

	DOM manuel (Partie 2)	**React** (Partie 3)
Créer une carte	`createElement` + `innerHTML` + `appendChild`	`<UserCard user={user} />`
Mettre à jour	Vider le conteneur + recréer tout	Modifier l'état, React met à jour tout seul
Événement clic	`addEventListener("click", fn)`	`onClick={fn}`
Données	Variables globales dans une IIFE	`useState` — état géré par React
État de chargement	Il faut le gérer manuellement	`chargement` + texte conditionnel

React est plus lisible, plus maintenable, et gère automatiquement les mises à jour du DOM. C'est pour cela qu'il est utilisé par Facebook, Airbnb, Netflix et des milliers d'entreprises.

5 Aide-mémoire

Concept	**Syntaxe / Exemple**
Fonction déclarée	`function double(x) { return x * 2; }`
Fonction expression	`const double = function(x) { return x * 2; };`
Fonction fléchée	`const double = x => x * 2;`
fetch + .then()	`fetch(url).then(r => r.json()).then(data => ...)`
fetch + async/await	`const r = await fetch(url); const d = await r.json();`
Créer un projet React	`npm create vite@latest nom -- --template react`
Lancer le serveur	`npm run dev`
État React	`const [val, setVal] = useState(initial);`
Composant React	`const MonComposant = () => <div>...</div>;`
Passer des données	`<Enfant nom={variable} />`
Boucle en JSX	`tableau.map(item => <Elem key={item.id} />)`

6 Résumé et conclusion

Dans ce lab, vous avez :

1. Découvert les **trois façons d'écrire une fonction** en JavaScript et refactorisé le starter avec des fonctions fléchées.
2. Compris le concept **d'asynchrone** et utilisé `fetch` + `async/await` pour récupérer des données depuis une API.
3. Créé votre **première application React** avec Vite, utilisé `useState` pour gérer l'état, et construit un composant réutilisable.

Voici où vous en êtes dans la progression du cours :

Étape	Statut	Ce que vous savez faire
DOM & événements	✓	Sélecteurs, événements, modification du DOM
Fonctions JS	✓	Déclaration, expression, fléchée, *hoisting*
Asynchrone	✓	`fetch`, `async/await`, gestion d'erreurs
React — les bases	✓	Composant, JSX, `useState`, props, `.map()`
React — aller plus loin	→ Prochain lab	`useEffect`, formulaires, routing

Et ensuite ?

Vous avez maintenant les bases pour construire de vraies interfaces utilisateur avec React. Dans les prochains labs, vous apprendrez à gérer les **formulaires**, le **routing** (navigation entre pages), et les **effets de bord** (`useEffect`) pour charger des données automatiquement au démarrage. Chaque concept s'appuie sur ce que vous avez appris aujourd'hui : les fonctions fléchées, l'asynchrone et les composants.

Exercice bonus — Pour aller plus loin

Bonus : ajouter un champ de recherche

Si vous avez terminé en avance, ajoutez un champ de recherche qui filtre les utilisateurs par nom. Indice :

1. Ajoutez un état `const [recherche, setRecherche] = useState("");`
2. Ajoutez un `<input>` avec `onChange={e => setRecherche(e.target.value)}`
3. Filtrez les utilisateurs :

```
users.filter(u => u.name.toLowerCase()
.includes(recherche.toLowerCase()))
```

4. Passez le tableau filtré au `.map()` au lieu de `users`

Cet exercice combine l'état, les événements et le rendu conditionnel.

Lab — Des fonctions JS à React — Terminé ✓

Vous savez écrire des fonctions modernes, récupérer des données depuis une API et créer une application React.

Prochaine étape : `useEffect`, formulaires et routing.

Chaque composant commence par une fonction fléchée.